

LOS GRISES JR

Software Architecture & Mathematical Control Model

WRO 2026 • Future Engineers

1. Software Architecture

The Los Grises Jr software follows the team's continuous engineering improvement methodology. Each subsystem is repeatedly tested, analyzed, modified, and optimized with three primary objectives: increasing reliability, improving repeatability, and simplifying maintenance.

The current architecture is organized around the following functional modules:

- Sensor acquisition — ultrasonic distance measurements.
- Sensor fusion — combination of multiple ultrasonic readings into a lateral control error.
- Obstacle detection — front ultrasonic sensor used as a safety condition.
- Motion control — closed-loop PID controller for steering correction.
- Speed control — dynamic calculation of the VEL variable.
- Motor control — application of forward and corrective motor commands.
- Safety override — stop and reverse maneuver when an obstacle is detected.

2. Software Execution Flow

START

Initialize Motor D (PCB adapter)

WHILE program is running:

Read front ultrasonic sensor

ARD $\leftarrow$ front distance

IF ARD < 25 cm:

Stop motors

Wait a few milliseconds

Reverse 0.5 motor rotations

ELSE:

Read the four lateral ultrasonic sensors

Side_A $\leftarrow$ Ultrasonic_1 + Ultrasonic_2

Side_B $\leftarrow$ Ultrasonic_3 + Ultrasonic_4

ERROR $\leftarrow$ Side_A - Side_B

Calculate / update VEL

Apply PID control using:

ERROR

VEL

Kp, Ki, Kd

Apply motor command

END IF

```
END WHILE  
END
```

3. Ultrasonic Sensor Fusion

The lateral control signal is generated from four ultrasonic sensors rather than a single measurement. Two sensor readings are combined for each side of the robot. The difference between the two groups provides the feedback signal used by the controller.

```
Side_A = Ultrasonic_1 + Ultrasonic_2  
Side_B = Ultrasonic_3 + Ultrasonic_4  
  
ERROR = Side_A - Side_B
```

This architecture increases the amount of information available to the controller and makes the lateral error less dependent on an individual sensor reading.

4. Error Calculation

The variable ERROR represents the difference between the two sensor groups. Its sign indicates which side has the larger combined measured distance, while its magnitude represents the degree of lateral imbalance.

```
ERROR = (US1 + US2) - (US3 + US4)
```

The resulting error is supplied to the PID controller as the primary feedback variable.

5. PID Motion Controller

Normal driving is controlled by a closed-loop Proportional-Integral-Derivative (PID) controller. The controller continuously reacts to the ultrasonic error and generates a correction for the motor commands.

```
Integral(t) = Integral(t-1) + ERROR(t)  
  
Derivative(t) = ERROR(t) - ERROR(t-1)  
  
PID_Output =  
    Kp × ERROR  
    + Ki × Integral  
    + Kd × Derivative
```

The final Kp, Ki, and Kd values are the result of repeated real-track testing and tuning. They were refined through multiple iterations instead of being selected only from theoretical calculations.

6. Dynamic Speed Control

The variable VEL defines the forward-speed component of the normal driving command. Its value is calculated by a dedicated control section so that forward velocity can be managed independently from the steering correction produced by the PID controller.

Because the uploaded screenshots do not provide enough resolution to verify every numerical threshold in the VEL block, this document intentionally does not invent or alter those values. The exact block logic should be transcribed directly from the original program when the high-resolution image is available.

7. Front Ultrasonic Safety System

A dedicated front ultrasonic measurement acts as a safety override. When ARD falls below 25 cm, normal forward control is interrupted.

```
IF ARD < 25 cm:
    Stop motors
    Wait a few milliseconds
    Reverse 0.5 rotations
ELSE:
    Continue PID-controlled driving
```

The emergency routine has priority over the normal PID motion command.

8. Motor Kinematics & Linear Displacement

The team's mathematical model uses the physical dimensions of the actual LEGO SPIKE chassis and thin wheels. The documented wheel diameter is 56.0 mm and the wheel circumference is 175.93 mm.

$$\begin{aligned}\text{Wheel_Circumference} &= \pi \times D \\ &\approx 175.93 \text{ mm}\end{aligned}$$

$$\begin{aligned}\text{Linear_Distance} &= \\ &(\text{Motor_Angle} / 360^\circ) \times 175.93 \text{ mm}\end{aligned}$$

$$\text{Linear_Displacement_per_Degree} \approx 0.4887 \text{ mm}/^\circ$$

The inverse relationship can be used to determine the motor rotation required for a target linear displacement:

$$\text{Motor_Angle} = \text{Distance(mm)} / 0.4887$$

9. Reverse Maneuver Analysis

The obstacle response commands a reverse movement of 0.5 rotations. Assuming the motor rotation is directly transmitted to a 56 mm wheel without gearing losses or slip, the ideal wheel displacement is:

$$\begin{aligned}\text{Reverse_Distance} &= \\ &0.5 \times 175.93 \text{ mm} \\ &\approx 87.97 \text{ mm}\end{aligned}$$

This is an ideal kinematic value. Real displacement can differ because of tire deformation, wheel slip, surface friction, drivetrain losses, and mechanical tolerances.

10. Motor Torque

Motor torque should be documented from the actual LEGO motor specification or, preferably, from measurements under the team's operating conditions. Torque is the rotational equivalent of force and is expressed as:

$$\tau = F \times r \times \sin(\theta)$$

For a wheel-driven system where the applied force is tangential to the wheel, $\sin(\theta) \approx 1$, giving:

$$\tau \approx F \times r$$

The total tractive force at the wheel can therefore be related to motor torque by:

$$F \approx \tau / r$$

A precise numerical torque value has intentionally not been inserted here because the supplied documentation does not specify the measured or manufacturer-rated torque of the Medium LEGO motors under the team's actual operating conditions. Adding an unsupported value would make the engineering report less rigorous.

11. Steering Geometry

The team's mathematical documentation specifies a wheelbase of 24.0 cm and a track width of 18.0 cm. These dimensions are used to model the steering geometry.

$$\delta_{\text{internal}} = \text{atan}\left(L / \left((L / \tan(\alpha)) - W/2 \right) \right)$$

$$\delta_{\text{external}} = \text{atan}\left(L / \left((L / \tan(\alpha)) + W/2 \right) \right)$$

The documented model gives different internal and external steering angles for a given central steering angle, which is consistent with the geometric requirements of an Ackermann-type steering model.

12. Diagonal Ultrasonic Geometry

For ultrasonic measurements taken at 45°, the team's model decomposes the measured diagonal distance into lateral and forward components.

$$\begin{aligned} Y_{\text{wall}} &= D_{\text{diagonal}} \times \sin(45^\circ) \\ &= D_{\text{diagonal}} \times 0.7071 \end{aligned}$$

$$\begin{aligned} X_{\text{forward}} &= D_{\text{diagonal}} \times \cos(45^\circ) \\ &= D_{\text{diagonal}} \times 0.7071 \end{aligned}$$

13. Engineering Decisions

- Use multiple ultrasonic sensors to generate a combined lateral error.
- Use closed-loop PID control so the robot can continuously correct deviations.
- Give the front obstacle detector priority over normal motion control.
- Separate forward speed (VEL) from steering correction.
- Tune controller parameters through repeated real-track testing.
- Use the measured physical dimensions of the actual robot in the mathematical model.
- Avoid unsupported numerical assumptions in the engineering documentation.

14. Testing & Validation

Controller development followed an iterative test cycle: implement, test on the real track, observe behavior, modify parameters or logic, and test again. The final PID configuration is therefore the result of empirical validation under real operating conditions.

15. Future Improvements

- Automatic ultrasonic offset calibration.
- Digital filtering or sensor averaging to reduce measurement noise.

- Adaptive PID parameters based on driving conditions.
- Automatic VEL optimization.
- Data logging for post-run PID analysis.
- Direct measurement of motor torque and drivetrain efficiency.
- Slip and friction compensation in the kinematic model.
- Automated parameter comparison between track runs.

16. Repository Structure

```
Los-Gris-Jr/  
├── README.md  
├── src/  
│   ├── main  
│   ├── ultrasonic  
│   ├── sensor_fusion  
│   ├── pid_controller  
│   ├── speed_control  
│   ├── obstacle_detection  
│   └── motor_control  
├── calibration/  
├── documentation/  
│   ├── software_architecture  
│   ├── mathematical_model  
│   ├── pid_tuning  
│   └── engineering_decisions  
└── tests/  
    └── track_testing
```

17. Reference Values from the Team's Mathematical Model

Parameter	Value	Purpose
Wheelbase (L)	24.0 cm	Steering / kinematic model
Track width (W)	18.0 cm	Steering geometry and stability
Wheel diameter	56.0 mm	Linear displacement
Wheel circumference	175.93 mm	Distance per revolution
Motor rotation resolution	0.4887 mm/°	Odometer conversion
Front critical distance	25 cm	Obstacle safety override
Reverse command	0.5 rotations	Obstacle recovery